

Marching Cubes in Fortran

Matthew J. Doyle

University of Bristol.

(Dated: October 22, 2019)

INTRODUCTION

This document should serve as a user manual for the `marching_cube` subroutine. Marching cubes is an algorithm for constructing an isosurface from a 3D mesh of points one box at a time. The context of use for the algorithm here is to visualise the Fermi surface of a simple cubic lattice for a given Fermi energy. This could be expanded to render other types of lattice. As the simple cubic structure has been used here, the boxes must have the geometry of cubes.

The implementation of Marching Cubes that has been used is known as Chernyaev's Marching Cubes 33, an improvement upon the original implementation which is plagued by unresolved ambiguities [1]. This method is known to provide a topically accurate result where the original does not.

METHOD

If the isosurface is represented by $F(x, y, z) = \alpha$, where α is the isovalue, then α can be subtracted from all nodes across the grid giving $F(x, y, z) = 0$ [1]. The modified nodes of any box can then be labelled as either positive or negative. Hence, positive nodes lie outside the surface and negative nodes lie inside.

The cubes are processed individually, therefore, the isosurface can be represented as a trilinear variation over the cube:

$$\begin{aligned} F(q, s, t) = & F_{000}(1 - q)(1 - s)(1 - t) + F_{100}q(1 - s)(1 - t) \\ & + F_{010}(1 - q)s(1 - t) + F_{110}qs(1 - t) \\ & + F_{001}(1 - q)(1 - s)t + F_{101}q(1 - s)t \\ & + F_{011}(1 - q)st + F_{111}qst \end{aligned}$$

where q, s, t are local coordinates of the cube (each varies 0 to 1), and the subscripts of F represent the values at each corner cube node [1]. It is possible to say that two nodes of the same sign are joined if there exists a path between them where the trilinear function does not change sign [1], if not we say these nodes are separated.

Face Ambiguity

Ambiguity on a face arises when two boxes which share the same face show a hole in the isosurface on this shared face. This occurs only when a face two negative and two positive diagonally opposed nodes. The problem is then to decide which two nodes are joined and which are separated.

An ambiguous face may be resolved by using a bilinear variation of F [2] thus F^{-1} must be a hyperbola on the face. If the positive nodes are labelled A, C and negative ones B, D then if $AC - BD > 0$ then the centre of the hyperbola is positive, thus the positive nodes are joined. The opposite is true for $AC - BD < 0$. However, some configurations require the opposite of the result (in which case the result is inverted by multiplying by a negative **face_number** which is taken from the test tables) and if A is negative then the sign of the hyperbola will be opposite also [3]. Therefore, in the function **face_test** we encode this by returning 1 if **face_number** $A(AC - BD) > 0$ and returning 0 if the opposite is true.

Internal Ambiguity

Once ambiguity along the face is resolved it is then possible to consider all nodes of the box and discover which are joined or separated. One method is to perform a bilinear variation of F over a plane parallel to a face as in [1] and check the three inequalities in Chernyaev's paper. If all three are true then the positive areas of the cube are internally joined, otherwise they

are separated. These tests are performed by **interior_ambiguity_test** and **test_interior** calls this function for specific cases of ambiguous faces.

Procedure

The algorithm uses four simple steps to process each box, as proposed by Thomas Lewiner [3].

1. Determine the case number and configuration
2. Check which faces need to be tested (given by face test tables) for said case number and configuration
3. Find the sub-case corresponding to the results of the face tests
4. Look-up the tiling edge codes for this sub-case and plot the corresponding vertices as triangles

MARCHING_CUBE

The **Marching_Cube subroutine** is called within a loop, over all boxes, in Bisection if the condition that the box is cut by the isosurface across more than three lines. Important arguments for this routine are: **poiplane** (contains the crossing vertices), **vert.edge** (contains which edge of the cube is cut by each crossing vertex), **eigvals** (an array of eigenvalues for each corner in the mesh) and **IKP** (indices to pick out the relevant corner eigenvalues from eigvals).

Initially, the cube corner coordinates are calculated, then the eigenvalues corresponding to each corner are extracted from **eigvals** using **IKP** as the index. The crossing vertices are then re-ordered into **vertlist** so that a vertex that sits on the 7th edge of the box is positioned in the 7th element of **vertlist**.

An 8-bit index is formed [4] to determine **caseindex** which increases for every corner value lower than the isovalue, **EF**. **Caseindex** is then used to look-up the case number, **casen**, and configuration, **config**, from the first table: cases. Depending on the value of **casen** given, different tests are applied to the cube, to determine which elements of the tiling tables should be used to draw triangles, these elements are placed into **trig** and then passed into the **add_triangle subroutine**.

ADD_TRIANGLE

This routine loops over all the triangles that can be drawn using the information given in **trig** and **vertlist**, one triangle at a time. Initially, the Fermi-velocity of each vertex is calculated so that it can be plotted with the mesh as the colour map.

If an entry in **trig** is 12 then the central vertex, **v12**, takes the place of one corner of the triangle. This vertex is the mean of all the crossing vertices in the cube.

Each vertex of the triangle (and corresponding Fermi velocity) are written to a file which is prefaced with the value of Fermi energy used and ends with "isosurf.dn". The format for this is for plotting a 3D mesh with **GNUplot**, however, this can easily be changed to work with another choice of plotting program.

The final step in the loop is to calculate the area of the triangle and add this to the array **kweight** into the correct element, which is determined by **linplane**.

LOOK-UP TABLES

The tiling tables contain rows of integers which specify how to draw the triangles for each specific case and sub-case. The integers themselves represent the edge which in turn will choose which elements from **vertlist** to use for each triangle. Note: these integers range from 0 to 12, with 12 representing the central vertex, so it is important to add 1 to each integer in **add_triangle** to select the correct element of **vertlist** since Fortran lists start at 1 (the tables were originally made for C++). The labelling of the cubes corresponding to the look-up tables used is shown in Figure 1.

The test tables contain the information about which faces to test for each case and configuration. Additionally, they contain information to determine the **subconfig** of case 13 in a similar fashion to how **caseindex** is determined in **MARCHING_CUBE**. The cases table has 256 entries, same as the original marching cubes algorithm, but instead of containing the tiling information it contains the case number, **casen** and configuration, **config**. These two variables are crucial to deciding which tests to perform along which face/edge in the ambiguous cases. The tables used correspond to Lewiner's implementation of the Marching Cubes algorithm and can be found at [5].

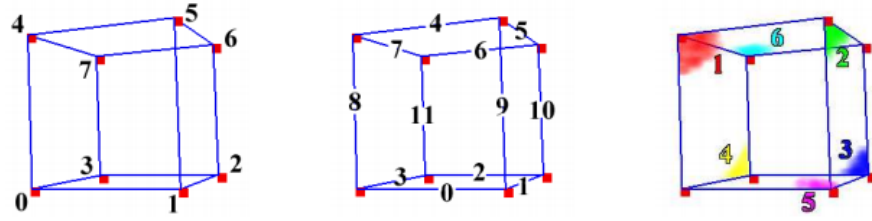


FIG. 1. The labelling of vertices, edges and faces for the look-up tables taken from [3]. Node 0 has the lowest x, y, z coordinates and node 6 the highest.

PLOTTING

The isosurface is output from the program as "isosurf.dn" and is compatible with plotting a 3D surface in GNUplot. Some examples of the output for different Fermi-energies are illustrated in Figure 2.

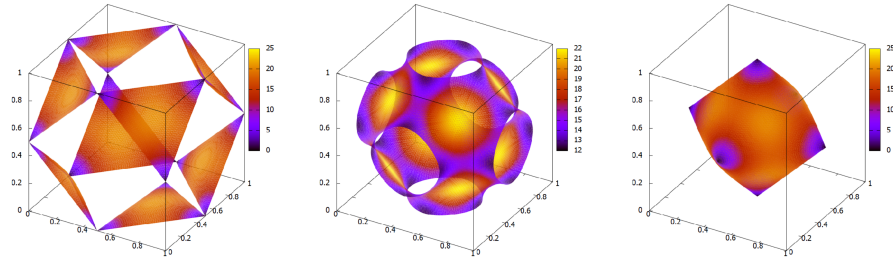


FIG. 2. The Fermi-surfaces rendered for $EF=2, 0, -2$ from left to right. The heat-map represents the length of the Fermi-velocity.

These commands can be simply copy and pasted into the **GNUplot** software to achieve plots such as in Figure 2:

```
> set pm3d
unset surface
set view equal xyz
set border 4095
set ticslevel 0
set datafile fortran
sp "isosurf.dn" u 1:2:3:4
```

The **density of states** as a function of EF can be plotted easily in any software using the information in the file **DOS** created as the final output for the program.

REFERENCES

- [1] *Marching Cubes 33: Construction of Topologically Correct Isosurfaces* Evgeni V. Chernyaev, No. CERN-CN-95-17. (1995).
- [2] *The Asymptotic Decider: Resolving the Ambiguity in Marching Cubes*, G.M.Nielson and B.Hamann, Proceedings of Visualization '91, IEEE Computer Society Press, pp. 83-90, 1991.
- [3] *Efficient implementation of Marching Cubes' cases with topological guarantees* Thomas Lewiner et al, Journal of graphics tools 8(2) pp.1-15 (2003)
- [4] *Polygonising a scalar field*, Paul Bourke (1994) <http://paulbourke.net/geometry/polygonise/> Last accessed 21/10/2019
- [5] *LookUpTable for the MarchingCubes 33 Algorithm*, Thomas Lewiner, (12/08/2002) https://github.com/liscustodio/modified_mc33/blob/master/source/LookUpTable.h Last accessed: 21/10/2019